

1. Encapsulation

Definition:

- Encapsulation is the bundling of data (attributes) and methods (functions) that operate on the data into a single unit, or class. It restricts direct access to some of the object's components, which is a means of preventing unintended interference and misuse of the methods and data.

Example:

```
java
VerifyOpen In EditorRunCopy code
1class Person {
2    private String name; // Private attribute
3
4    // Getter method
5    public String getName() {
6        return name;
7    }
8
9    // Setter method
10   public void setName(String name) {
11       this.name = name;
12   }
13}
14
15public class Main {
16   public static void main(String[] args) {
17       Person person = new Person();
18       person.setName("Alice");
19       System.out.println("Name: " + person.getName());
20   }
21}
```

Output:

```
VerifyOpen In EditorRunCopy code
1Name: Alice
```

2. Inheritance

Definition:

- Inheritance is a mechanism where a new class (subclass) can inherit the properties and methods of an existing class (superclass). This promotes code reusability and establishes a relationship between classes.

Example:

```
java
VerifyOpen In EditorRunCopy code
1// Superclass
2class Animal {
3    void eat() {
4        System.out.println("This animal eats food.");
5    }
6}
7
8// Subclass
9class Dog extends Animal {
```

```

10    void bark() {
11        System.out.println("The dog barks.");
12    }
13}
14
15public class Main {
16    public static void main(String[] args) {
17        Dog myDog = new Dog();
18        myDog.eat(); // Inherited method
19        myDog.bark(); // Subclass method
20    }
21}

```

Output:

VerifyOpen In EditorRunCopy code
1This animal eats food.
2The dog barks.

3. Polymorphism

Definition:

- Polymorphism allows methods to do different things based on the object it is acting upon, even if they share the same name. It can be achieved through **method overloading** (compile-time polymorphism) and **method overriding** (runtime polymorphism).

Example of Method Overloading:

```

java
VerifyOpen In EditorRunCopy code
1class Calculator {
2    // Method to add two integers
3    int add(int a, int b) {
4        return a + b;
5    }
6
7    // Method to add three integers
8    int add(int a, int b, int c) {
9        return a + b + c;
10   }
11}
12
13public class Main {
14    public static void main(String[] args) {
15        Calculator calc = new Calculator();
16        System.out.println("Sum of two numbers: " + calc.add(5, 10)); //
Output: 15
17        System.out.println("Sum of three numbers: " + calc.add(5, 10, 15)); //
Output: 30
18    }
19}

```

Example of Method Overriding:

```

java
VerifyOpen In EditorRunCopy code
1class Animal {
2    void sound() {
3        System.out.println("Animal makes a sound");

```

```

4    }
5}
6
7class Cat extends Animal {
8    @Override
9    void sound() {
10        System.out.println("Cat meows");
11    }
12}
13
14public class Main {
15    public static void main(String[] args) {
16        Animal myAnimal = new Cat(); // Upcasting
17        myAnimal.sound(); // Calls the overridden method
18    }
19}

```

Output:

```

VerifyOpen In EditorRunCopy code
1Cat meows

```

4. Abstraction

Definition:

- Abstraction is the concept of hiding the complex implementation details and showing only the essential features of the object. It can be achieved using **abstract classes** and **interfaces**.

Example of Abstract Class:

```

java
VerifyOpen In EditorRunCopy code
1abstract class Shape {
2    abstract void draw(); // Abstract method
3}
4
5class Circle extends Shape {
6    void draw() {
7        System.out.println("Drawing a Circle");
8    }
9}
10
11public class Main {
12    public static void main(String[] args) {
13        Shape myShape = new Circle();
14        myShape.draw(); // Output: Drawing a Circle
15    }
16}

```

Example of Interface:

```

java
VerifyOpen In EditorRunCopy code
1interface Drawable {
2    void draw(); // Abstract method
3}
4
5class Rectangle implements Drawable {
6    public void draw() {
7        System.out.println("Drawing a Rectangle");
8    }

```

```
9}  
10  
11public class Main {  
12    public static void main(String[] args) {  
13        Drawable myRectangle = new Rectangle();  
14        myRectangle.draw(); // Output: Drawing a Rectangle  
15    }  
16}
```

Summary

- **Encapsulation** protects the data by restricting access.
- **Inheritance** allows for code reuse and establishes a relationship between classes.
- **Polymorphism** enables methods to behave differently based on the object.
- **Abstraction** hides complex implementation details and exposes only the necessary parts.

These OOP concepts are essential for building scalable and maintainable applications in Java.

Understanding and applying these principles will greatly enhance your programming skills. If you have any further questions or need more examples, feel free to ask!